

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
 Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	Athavan K	Reg. No.:	192524036
Section / Batch:	2025	Date:	16/7/26

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Q1. Autonomous Drone Coordination System

Q2. Smart Manufacturing Quality Control System

Code of Conduct

I Athavan K(192524036) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Athavan K

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						
Faculty In-charge		Course Coordinator		Head of Department		
Signature: _____		Signature: _____		Signature: _____		
Date: _____		Date: _____		Date: _____		

Question 58 — Autonomous Drone Coordination System

An autonomous drone coordination system manages flight data using the recurrence $T(n) = 2T(n/3) + n^2$. Apply the Master Theorem to determine the time complexity. Clearly identify parameters and justify the selected case. Analyze how coordination performance changes with increasing drone count. Interpret efficiency using asymptotic notation.

1. Problem Restatement

A fleet of autonomous drones must coordinate flight paths, avoid collisions, and synchronise position data. The coordination engine repeatedly splits the fleet's flight-data set into two smaller groups, each one-third the size of the original set, and processes each group recursively. In addition to the recursive calls, the algorithm performs quadratic-time work at every level of recursion — this quadratic component typically arises from pairwise comparisons between drones for collision detection, trajectory conflict resolution, or all-pairs distance computation within a group. This behaviour is captured precisely by the recurrence:

$$T(n) = 2 \cdot T(n/3) + n^2$$

Here, n represents the number of drones (or flight-data records) being coordinated. The goal of this analysis is to determine the exact asymptotic growth rate of $T(n)$ using the Master Theorem, and to interpret what that growth rate means for the scalability of the coordination system as drone-fleet size increases.

2. The Master Theorem — General Form

The Master Theorem provides a direct method for solving divide-and-conquer recurrences of the standard form:

$$T(n) = a \cdot T(n/b) + f(n), \text{ where } a \geq 1 \text{ and } b > 1$$

In this general form, a is the number of recursive subproblems generated at each call, b is the factor by which the problem size shrinks in each subproblem, and $f(n)$ is the cost of the work done outside the recursive calls — that is, the cost of dividing the problem and combining the partial results. The theorem compares the growth of $f(n)$ against the "watershed" function $n^{\log_b a}$, which represents the total cost contributed by the recursive branching structure alone (ignoring the extra work $f(n)$). Depending on which of the two functions dominates, one of three cases applies:

- Case 1: If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, the recursive branching dominates and $T(n) = \Theta(n^{\log_b a})$.
- Case 2: If $f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$ for some $k \geq 0$, the two contributions are balanced and $T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$.
- Case 3: If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and the regularity condition $a \cdot f(n/b) \leq c \cdot f(n)$ holds for some constant $c < 1$ and all sufficiently large n , then the extra work at the root dominates and $T(n) = \Theta(f(n))$.

3. Identifying the Parameters

Comparing the given recurrence $T(n) = 2T(n/3) + n^2$ against the standard form $T(n) = aT(n/b) + f(n)$, the parameters are identified directly:

Parameter	Meaning	Value in this problem
a	Number of recursive subproblems created	2
b	Factor by which problem size shrinks	3
$f(n)$	Cost of work outside the recursive calls	n^2
n	Size of the input (number of drones)	n

Each recursive call handles a group that is one-third the size of the parent group, and exactly two such groups are formed at every stage. The n^2 term represents the additional coordination overhead — for instance, comparing every drone's trajectory against every other drone's trajectory within a group — performed before or after the two recursive calls return.

4. Step 1 — Compute the Watershed Exponent $n^{(\log_b a)}$

The critical exponent that separates the three Master Theorem cases is $\log_b a$. Substituting the identified values:

$$\log_b a = \log_3 2 \approx 0.6309$$

Therefore, the watershed function is $n^{(\log_b a)} = n^{0.6309}$, a sub-linear function that grows more slowly than n itself. This value represents the total cost that would result purely from the two-way, factor-of-three recursive branching if there were no extra work performed at each level.

5. Step 2 — Compare $f(n)$ with the Watershed Function

We must now compare $f(n) = n^2$ with $n^{(\log_b a)} = n^{0.6309}$. Since the exponent of $f(n)$ is 2, and the watershed exponent is approximately 0.6309, the difference between the two exponents is:

$$2 - 0.6309 = 1.3691 = \epsilon$$

Because this difference ϵ is a strictly positive constant, $f(n)$ grows polynomially faster than $n^{(\log_b a)}$. Formally, $f(n) = \Omega(n^{(\log_b a + \epsilon)})$ for $\epsilon \approx 1.3691$, which is precisely the condition required to test Case 3 of the Master Theorem.

6. Step 3 — Verify the Regularity Condition

Case 3 cannot be applied on the basis of the exponent comparison alone; the regularity condition $a \cdot f(n/b) \leq c \cdot f(n)$ must also hold for some constant $c < 1$ and all sufficiently large n . Substituting the values of this problem:

$$a \cdot f(n/b) = 2 \cdot (n/3)^2 = 2n^2/9 \approx 0.2222 n^2$$

We need this quantity to be bounded above by $c \cdot n^2$ for some $c < 1$. Choosing $c = 2/9 \approx 0.2222$ satisfies the inequality exactly, and since $2/9 < 1$, the regularity condition is confirmed to hold for all n . This step is essential: without verifying regularity, applying Case 3 would not be mathematically justified.

7. Step 4 — Case Selection and Justification

With both conditions of Case 3 satisfied — namely $f(n) = \Omega(n^{\log_b a + \epsilon})$ and the regularity condition $a \cdot f(n/b) \leq c \cdot f(n)$ — the Master Theorem guarantees:

$$T(n) = \Theta(f(n)) = \Theta(n^2)$$

The choice of Case 3 (rather than Case 1 or Case 2) is justified because the non-recursive work $f(n) = n^2$ dominates the cost contributed by the recursive subproblems. In other words, the coordination overhead performed at each level of recursion is asymptotically larger than the combined cost of all recursive calls beneath it, so the total running time is governed almost entirely by the work done at the root of the recursion tree.

8. Verification by Recursion Tree

The Case 3 result can be independently confirmed by expanding the recursion tree and summing the work performed at every level.

Level	No. of nodes	Size per node	Work per node	Total work at level
0	1	n	n^2	n^2
1	2	$n/3$	$(n/3)^2$	$2n^2/9 \approx 0.222 n^2$
2	4	$n/9$	$(n/9)^2$	$4n^2/81 \approx 0.049 n^2$
3	8	$n/27$	$(n/27)^2$	$8n^2/729 \approx 0.011 n^2$
...	...	...	...	...

The total work per level forms a geometric series with common ratio $r = 2/9$ (since each level's total is $2/9$ of the level above). Because $r < 1$, the series converges, and the sum of the entire infinite series is bounded by:

$$\text{Total work} = n^2 \cdot \sum (2/9)^i = n^2 \cdot [1 / (1 - 2/9)] = n^2 \cdot (9/7) \approx 1.286 n^2$$

Since the series converges to a constant multiple of n^2 , the total work across all levels of the tree is $\Theta(n^2)$ — the root level's contribution alone already accounts for the dominant share of the total cost, and every subsequent level contributes a strictly smaller, geometrically shrinking amount. This exactly matches the Case 3 conclusion derived from the Master Theorem, providing an independent cross-check of the result.

9. Scalability Analysis for the Drone Coordination System

The result $T(n) = \Theta(n^2)$ has direct practical significance for a drone-swarm coordination platform. Because the dominant cost lies in the root-level quadratic combine step (the pairwise trajectory and conflict-resolution comparisons), the benefit of dividing the fleet recursively into thirds becomes progressively less significant as the fleet grows. In practice this means:

- Doubling the number of drones being coordinated increases the total coordination time by roughly a factor of four, since the running time is proportional to n^2 .
- For small to moderate fleets, the recursive decomposition still reduces the effective working-set size, offering some memory and cache-locality benefits even though it does not change the asymptotic order.
- For very large fleets (hundreds to thousands of drones), the quadratic combine step becomes the dominant bottleneck, and the system's real-time responsiveness may degrade sharply unless the pairwise coordination step itself is replaced with a sub-quadratic technique, such as spatial partitioning (e.g. k-d trees or grid-based binning) to avoid all-pairs comparisons.
- The recursive splitting by a factor of three ($b = 3$) with only two branches ($a = 2$) means the branching factor a is smaller than b , so the recursion tree is relatively "thin" — this reduces recursive overhead but does nothing to offset the quadratic cost dominating from the top of the tree.

10. Interpretation Using Asymptotic Notation

The final result is expressed tightly as:

$$T(n) = \Theta(n^2)$$

This Θ (theta) bound indicates that n^2 is simultaneously an asymptotic upper bound ($O(n^2)$) and an asymptotic lower bound ($\Omega(n^2)$) on the running time — the algorithm's performance is neither better nor worse than quadratic growth for large n . In Big-O terms, an engineer would state that the coordination algorithm runs in $O(n^2)$ time in the worst case, and this bound is tight because it is also $\Omega(n^2)$, meaning no faster asymptotic characterisation is possible for this particular recurrence.

11. Conclusion

For the recurrence $T(n) = 2T(n/3) + n^2$, the Master Theorem's Case 3 applies because the combine-step cost $f(n) = n^2$ asymptotically dominates the recursive branching cost $n^{\log_3 2}$, and the regularity condition confirms that this dominance is stable across recursive levels. The resulting time complexity is $\Theta(n^2)$. For the autonomous drone coordination system, this means overall performance is governed by the pairwise coordination workload rather than by the depth or shape of the recursive decomposition — a critical insight for system designers seeking to scale the platform to very large drone fleets, since only a reduction in the asymptotic order of the combine step itself (not further recursive splitting) will meaningfully improve scalability.

Question 59 — Smart Manufacturing Quality Control System

A manufacturing quality control system inspects products using the recurrence $T(n) = 9T(n/3) + n$. Apply the Master Theorem to determine the time complexity. Identify parameters and justify the case selection. Analyze how inspection time scales with production volume. Interpret efficiency using asymptotic notation.

1. Problem Restatement

A smart manufacturing quality-control system inspects a batch of n products for defects. To do this efficiently, the batch is recursively divided into smaller sub-batches: at every recursive call, the current batch is split into nine sub-batches, each one-third the size of the parent batch, and each sub-batch is inspected recursively. In addition, a linear amount of overhead work — for example, coordinating inspection results, logging, or preprocessing sensor readings — is performed at each level before combining results. This process is captured by the recurrence:

$$T(n) = 9 \cdot T(n/3) + n$$

The objective is to apply the Master Theorem to determine the exact asymptotic time complexity of this inspection process and to interpret what the result implies for scaling quality control to higher production volumes.

2. The Master Theorem — General Form

As before, the Master Theorem applies to recurrences of the form $T(n) = aT(n/b) + f(n)$, where $a \geq 1$ is the number of recursive subproblems, $b > 1$ is the factor by which the input shrinks in each subproblem, and $f(n)$ is the cost of non-recursive work at each level. The theorem compares $f(n)$ with the watershed function $n^{\log_b a}$:

- Case 1: If $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$, the recursive branching dominates, giving $T(n) = \Theta(n^{\log_b a})$.
- Case 2: If $f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$ for $k \geq 0$, the branching and extra work are balanced, giving $T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$.
- Case 3: If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$, and the regularity condition $a \cdot f(n/b) \leq c \cdot f(n)$ holds for a constant $c < 1$, the extra work dominates, giving $T(n) = \Theta(f(n))$.

3. Identifying the Parameters

Matching $T(n) = 9T(n/3) + n$ to the standard form $T(n) = aT(n/b) + f(n)$ gives the following parameters:

Parameter	Meaning	Value in this problem
a	Number of recursive subproblems created	9
b	Factor by which problem size shrinks	3
$f(n)$	Cost of work outside the recursive calls	n

n	Size of the input (number of products)	n
---	--	---

At every recursive call, the batch of products is fragmented into nine smaller sub-batches, each a third of the size of the original — a comparatively wide (bushy) branching factor relative to how much the problem shrinks. The additional $f(n) = n$ term represents the linear-time bookkeeping, logging, or aggregation work performed once per call, outside of the nine recursive calls themselves.

4. Step 1 — Compute the Watershed Exponent $n^{(\log_b a)}$

The critical comparison exponent is $\log_b a$. Substituting $a = 9$ and $b = 3$:

$$\log_b a = \log_3 9 = 2$$

This is an exact integer value because $3^2 = 9$. Therefore, the watershed function is $n^{(\log_b a)} = n^2$, meaning that the recursive branching structure alone — even with no additional per-call work — would already contribute quadratic-order total cost, purely from the fact that nine subproblems are generated at each level while the size shrinks by only a factor of three.

5. Step 2 — Compare $f(n)$ with the Watershed Function

We now compare $f(n) = n$ with $n^{(\log_b a)} = n^2$. Since n grows strictly more slowly than n^2 , we can express this relationship as:

$$f(n) = O(n^{(2 - \epsilon)}) \text{ for } \epsilon = 1, \text{ since } n = O(n^1)$$

Because there exists a constant $\epsilon = 1 > 0$ for which $f(n) = O(n^{(\log_b a - \epsilon)})$ holds, this satisfies precisely the condition required for Case 1 of the Master Theorem. Unlike Case 3, Case 1 does not require verifying any additional regularity condition — the polynomial gap between $f(n)$ and $n^{(\log_b a)}$ is sufficient on its own.

6. Step 3 — Case Selection and Justification

Since $f(n) = n$ is polynomially smaller than $n^{(\log_b a)} = n^2$, the cost of the non-recursive combine step is asymptotically insignificant compared to the sheer volume of work generated by the recursive branching itself. Case 1 of the Master Theorem therefore applies directly, giving:

$$T(n) = \Theta(n^{(\log_b a)}) = \Theta(n^2)$$

This case is selected — rather than Case 2 or Case 3 — because the recursive subdivision into nine branches per call ($a = 9$) so heavily outweighs the shrink factor of three ($b = 3$) that the total cost accumulates almost entirely from the sheer number of subproblems generated near the bottom of the recursion tree, rather than from any work performed while combining results.

7. Verification by Recursion Tree

This conclusion can be verified independently by expanding the recursion tree level by level and tracking both the non-recursive work per level and the number of nodes.

Level	No. of nodes	Size per node	Work per node	Total work at level
0	1	n	n	n
1	9	n/3	n/3	$9 \cdot (n/3) = 3n$
2	81	n/9	n/9	$81 \cdot (n/9) = 9n$
3	729	n/27	n/27	$729 \cdot (n/27) = 27n$
...	...	...	...	...

Unlike the drone-coordination example, here the total work per level increases geometrically as we descend the tree, with common ratio $r = 3$ between successive levels (each level's total work is three times the level above it). This means the work is leaf-heavy: the deepest level of the tree contributes the overwhelming majority of the total cost. The recursion terminates when $n/3^k = 1$, i.e. at depth $k = \log_3 n$. At that final level, the number of leaf nodes is:

$$a^{(\log_b n)} = 9^{(\log_3 n)} = n^{(\log_3 9)} = n^2$$

Since there are $\Theta(n^2)$ leaves, each performing $\Theta(1)$ constant work at the base case, the total cost contributed by the leaves alone is already $\Theta(n^2)$ — and because the level-by-level totals grow geometrically toward the leaves, the sum across all levels is dominated by this final level, confirming the total running time is $\Theta(n^2)$, exactly matching the Case 1 result from the Master Theorem.

8. Scalability Analysis for the Quality Control System

The result $T(n) = \Theta(n^2)$ carries important implications for how a smart manufacturing quality-control system will behave as production volume increases. Because the total inspection cost is dominated by the sheer branching factor of the recursive decomposition rather than by the linear per-call overhead, the following observations apply:

- Doubling the number of products inspected increases the total inspection time by roughly a factor of four, since running time scales quadratically with n , despite the fact that each individual level of recursion only performs cheap, linear-time bookkeeping.
- The branching factor $a = 9$ being much larger than the shrink factor $b = 3$ means the recursion tree is very "bushy": the number of subproblems grows far faster than the rate at which each subproblem shrinks, producing an explosion of leaf-level base cases for large batches.
- Because the dominant cost comes from the sheer number of recursive calls (particularly near the leaves) rather than from the combine step, optimising the linear bookkeeping work $f(n) = n$ would have negligible impact on overall performance — the real lever for improving scalability is reducing the branching factor a , or restructuring the algorithm so that fewer, larger sub-batches are inspected per call.
- For very large production runs (e.g. tens of thousands of units per shift), this quadratic growth could become a genuine throughput bottleneck, motivating a redesign toward an algorithm with a

smaller effective branching factor, such as a binary or ternary split with only $\Theta(1)$ or $\Theta(\log n)$ subproblems per call rather than nine.

9. Interpretation Using Asymptotic Notation

The tight asymptotic bound obtained is:

$$T(n) = \Theta(n^2)$$

This means n^2 is simultaneously a valid upper bound ($O(n^2)$) and lower bound ($\Omega(n^2)$) on the running time of the inspection algorithm — its performance is fundamentally quadratic, and no asymptotically faster characterisation is possible given this particular recursive structure. From a systems perspective, this $\Theta(n^2)$ behaviour should be treated as equivalent in scalability terms to a naive, non-recursive quadratic-time inspection scheme, since the recursive decomposition does not, in this case, buy any asymptotic advantage.

10. Conclusion

For the recurrence $T(n) = 9T(n/3) + n$, the Master Theorem's Case 1 applies because the non-recursive overhead $f(n) = n$ is polynomially smaller than the watershed function $n^{(\log_3 9)} = n^2$, meaning the cost of recursive branching entirely dominates the total running time. The resulting complexity is $\Theta(n^2)$, independently confirmed through recursion-tree analysis showing that the leaves of the tree contribute the bulk of the total work. For the smart manufacturing quality-control system, this indicates that inspection time will scale quadratically with production volume, and that meaningful performance improvements must come from reducing the branching factor of the recursive decomposition rather than from optimising the lightweight per-call bookkeeping work.